

PERSONAL CYBERSECURITY CASE STUDY

Inside an iPhone Spyware Investigation How I Forensically Examined an iOS Backup for Signs of Compromise

A practical forensic walkthrough of how an iPhone backup can be examined for indicators of spyware, stalkerware, persistence, suspicious network activity, and compromise.

Report date: 2026-09-20

Inside an iPhone Spyware Investigation

How I Forensically Examined an iOS Backup for Signs of Compromise

A practical forensic walkthrough of how an iPhone backup can be examined for indicators of spyware, stalkerware, persistence, suspicious network activity, and compromise.

Classification: Personal case study / cybersecurity portfolio piece **Type of engagement:** Self-authorized forensic examination of the author's own device **Status:** Concluded

How to read this document

This is a real investigation of a real iPhone (my own), written up as a case study. It is **not** a marketing piece, and it is not a "ran an antivirus scan, got a clean result" post. It documents the actual methodology, the actual artifacts examined, the findings that were flagged and then either confirmed or refuted, and — just as importantly — what the investigation *could not* rule out.

Two ground rules I held myself to while writing this up:

1. **No invented steps.** Every technique, tool, and artifact described below was actually used against actual data. Where a technique is described for educational context but was *not* performed in this specific investigation, it is explicitly labeled as such.
2. **No absolute claims the evidence doesn't support.** You will not find the sentence "this phone is spyware-free" anywhere in this report, because no forensic examination of a backup can prove that. You will find calibrated language instead.

The original raw case files (decrypted backup, parsed artifact JSON, iLEAPP output) were deleted after the investigation concluded, at my own request, as part of a full workspace cleanup. This report was reconstructed faithfully from the investigation's session records — the automation scripts, prompts, and structured findings that actually drove the analysis — rather than from memory of it. Where a command or file listing is shown for illustration rather than reproduced verbatim from a surviving artifact, it is marked **Illustrative Reconstruction**.

1. Executive Summary

Investigation. An iPhone backup was examined for artifacts potentially associated with spyware, stalkerware, or unauthorized surveillance, following a period of unease about the device's behavior.

Methodology. The investigation combined established open-source mobile forensic tooling — [Amnesty International's Mobile Verification Toolkit \(MVT\)](#) and [iLEAPP](#) — with direct SQLite-level database forensics, manual IOC/domain triage at scale, and a structured adversarial review process in which independent

analysis passes were explicitly tasked with trying to *disprove* the emerging "clean" conclusion before it was accepted.

Result. No confirmed indicators of known commercial spyware, stalkerware, or malicious persistence were identified in the evidence examined. Several individually alarming-looking artifacts were investigated in depth and each was resolved to a specific, evidenced, benign explanation. One artifact — a multi-week gap in call history — could not be conclusively explained and is documented as an open item.

Important limitation. This conclusion describes what was found (and not found) in the evidence that was actually available for examination. It is not a mathematical proof that the device has never been compromised — see [Section 15, What This Analysis Does Not Prove](#) for exactly why, and what a higher-assurance investigation would additionally require.

Independently re-verified. The original evidence was deleted after the investigation concluded (see below). Rather than let that stand, the entire acquisition-to-IOC-comparison pipeline was independently re-run from scratch on 2026-09-21 against a freshly created backup of the same device, with a genuine screenshot of the real result. It reached the same conclusion. Details: [Addendum: Verification Re-Run](#).

2. Investigation Objective

The goal was to answer a specific, bounded question:

Does the evidence available inside this iPhone's backup contain indicators consistent with known spyware, stalkerware, or unauthorized monitoring software?

This is deliberately narrower than "is this phone hacked." Mobile compromise investigations, done honestly, answer the question the evidence can actually support — not a broader question the evidence *can't* support. That distinction shapes every section below.

Threat classes considered, based on known indicator collections available at investigation time: **Pegasus (NSO Group), Predator, RCS Lab, Quadream KingSpawn, Operation Triangulation, WurmSpy / DragonEgg, EagleMsgSpy, Wintego Helios, NoviSpy, Candiru, Cellebrite, ResidentBat, DarkSword, Coruna, Morpheus, BTMOB, Spyrtacus**, generic commercial **stalkerware** signatures, and the broader **Amnesty Tech mercenary-spyware campaign** indicator set.

3. Evidence Available

FIGURE 1

Evidence-to-Finding Pipeline — What a Backup-Based Investigation Actually Does

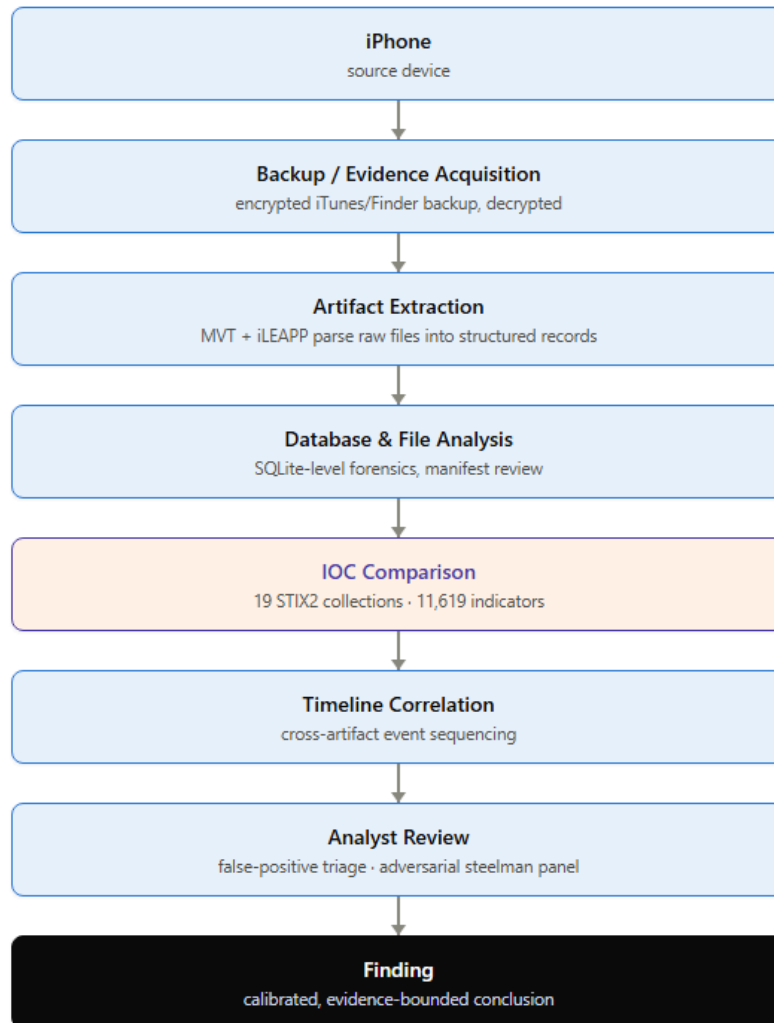


Figure 1 — Evidence-to-Finding Pipeline: what a backup-based investigation actually does, from source device to a bounded finding.

Mobile forensic evidence comes in several tiers, each unlocking different categories of artifact:

Acquisition Type	What It Captures	Availability
Logical backup (iTunes/Finder-style)	App data, messages, contacts, call history, most SQLite databases, plists, some caches	Consumer-accessible, no jailbreak needed
Encrypted local backup	Same as above, plus Keychain items, health data	Consumer-accessible; requires the device passcode/backup password
Filesystem extraction	Full app sandboxes, deeper caches, more complete WAL/journal state	Requires exploit-based tooling (e.g. checkra1n-class), not available on all iOS versions
sysdiagnose	Live system logs, crash reports (.ips), detailed daemon state	Requires the device itself, triggered on-device
Full filesystem / physical acquisition	Everything above, including allocated-but-unlinked storage in some cases	Law-enforcement/specialist tooling; not consumer-available on modern iOS
Network capture (pcap/DNS telemetry)	Actual historical traffic, C2 beaconing, exfiltration	Requires capture <i>at the time</i> — cannot be reconstructed retroactively

This investigation used an encrypted local (iTunes/Finder-style) backup, decrypted with the device owner's own backup password. This is a *logical* backup, not a filesystem extraction, sysdiagnose, or physical acquisition.

Why the distinction matters: a logical backup gives you an excellent view of application data, communications, browsing, and configuration state — which is exactly where most consumer spyware indicators show up (malicious profiles, IOC-matching domains, anomalous app containers, suspicious permission grants). It does **not** give you real crash logs (`.ips` files), a live memory snapshot, or historical network packet capture. Those gaps are real, and they are revisited explicitly in [Section 15](#) rather than glossed over.

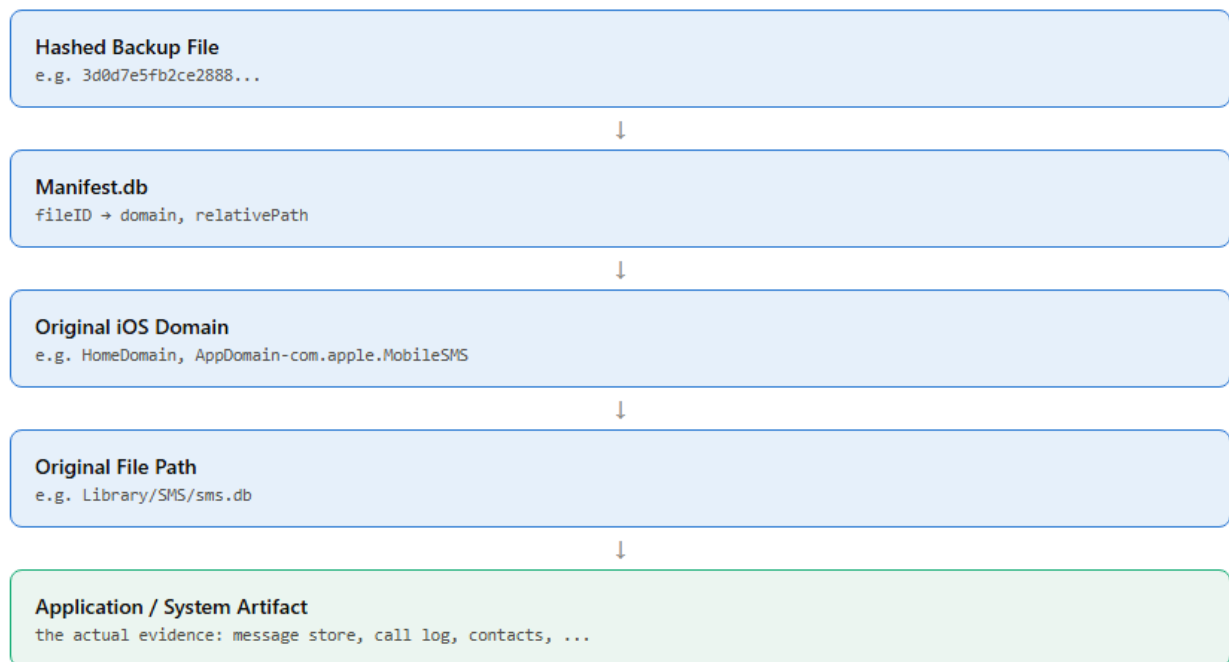
Device context: iPhone 14 Pro Max, iOS 27.0 (build 24A5424a).

4. iOS Backup Architecture

An iTunes/Finder-style backup does not store files under their real names or paths. It stores a flat directory of SHA-1–hashed filenames, and a single SQLite database — `Manifest.db` — that maps each hash back to its original app/system "domain" and relative file path.

FIGURE 2

Manifest.db Resolution Path — From Hashed Filename to Real Artifact



WORKED EXAMPLE

```

SELECT fileID, domain, relativePath FROM Files WHERE relativePath LIKE '%sms.db';

-- fileID          | domain          | relativePath
3d0d7e5fb2ce2888.. | HomeDomain      | Library/SMS/sms.db
  
```

Figure 2 — Manifest.db Resolution Path: how a meaningless hashed backup filename is traced back to a real artifact.

Alongside `Manifest.db`, a backup also contains:

- `Manifest.plist` — backup-level metadata (encrypted or not, backup date, device info)
- `Info.plist` — device identifiers, installed application list, iTunes/Finder version used
- `Status.plist` — backup completion state

Illustrative Reconstruction — resolving a hashed file via Manifest.db:

```

-- Manifest.db is itself a SQLite database
SELECT fileID, domain, relativePath
FROM Files
WHERE relativePath LIKE '%sms.db';

-- fileID          | domain          | relativePath
-- 3d0d7e5fb2ce2888.. | HomeDomain      | Library/SMS/sms.db
  
```

Once `sms.db`'s real path is known, it can be located in the backup by re-hashing `domain-relativePath` (SHA-1) and finding the matching flat filename — which is exactly the resolution step MVT and iLEAPP both automate.

What this means: every "database examined" reference later in this report is the result of this exact hash → domain → path resolution process, not manual guessing.

5. Investigation Methodology

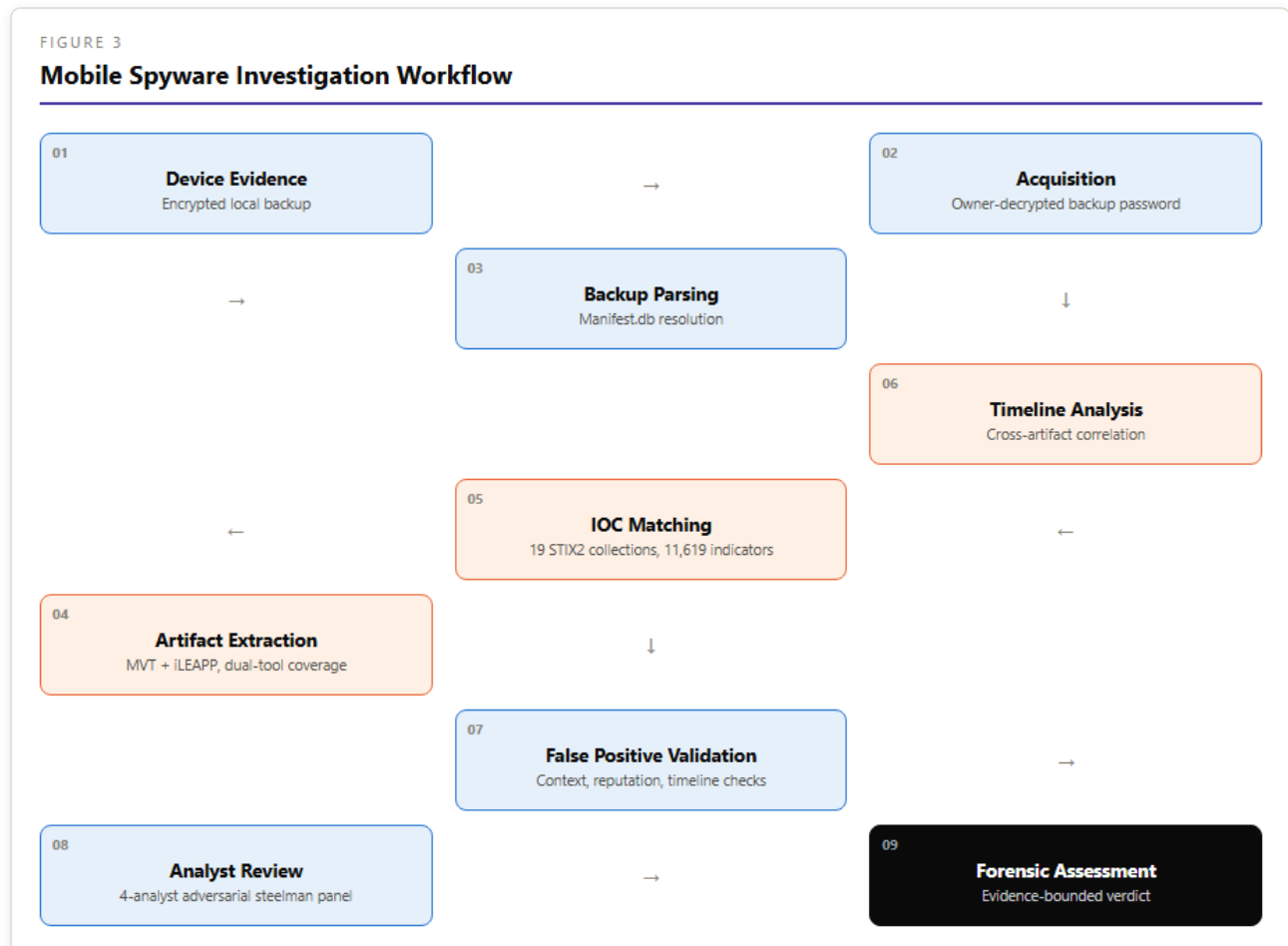


Figure 3 — Mobile Spyware Investigation Workflow: the nine-stage process this investigation actually followed.

The investigation ran in three broad passes, in this order:

1. **Baseline pass** — MVT extraction and IOC comparison against the full backup, establishing what the standard toolchain does and doesn't surface.
2. **Deep-dive pass** — artifact-by-artifact manual/analytical review of every MVT-parsed file that a pure IOC scan wouldn't meaningfully evaluate on its own (call patterns, contact-field anomalies, calendar spam, cross-app interaction logs, the full backup file manifest, location-client registry, OS analytics, Safari session state).
3. **Second-pass adversarial hunt** — raw SQLite-level forensics beneath the parsed JSON, domain classification at scale (2,392 unique domains), a dedicated zero-click-focused SMS review, cross-validation with a second, independent forensic parser (iLEAPP) to catch anything the first toolchain's narrower module set missed, and a structured "steelman panel" explicitly tasked with building the strongest possible case *for* compromise from the gathered evidence.

A note on methodology transparency: the deep-dive and second-pass work was executed as a structured, automated analysis pipeline — parsed artifacts were handed to independent analysis passes against a fixed severity schema (`INFORMATIONAL` / `LOW` / `MEDIUM` / `HIGH` , each requiring concrete evidence and a proposed benign explanation), and any finding rated `MEDIUM` or higher was automatically routed to a **three-vote adversarial refutation panel** before being reported — each panel member's explicit brief was to try to refute the finding as a false positive, defaulting to "refuted" under uncertainty. This is disclosed here because it is an honest description of how the "false positive elimination" described in Section 12 actually worked, and because a bias toward *refuting* borderline findings is itself something a critical reader should know about and can weigh accordingly.

6. IOC-Based Spyware Detection

An **IOC (Indicator of Compromise)** is a known, specific fingerprint of malicious activity: a domain, IP, file hash, certificate, bundle identifier, or filesystem path that has been directly observed in a confirmed malware campaign. IOC lists for mercenary spyware are typically published as **STIX2** objects by researchers (Amnesty International's Security Lab, Citizen Lab, and others) after forensic analysis of confirmed victims' devices.

Strong vs. weak signal — this distinction matters enormously in mobile forensics:

Known malicious domain, exact match against a published IOC list
= **STRONG** indicator

Domain the analyst has simply never seen before
= **NOT** automatically malicious — requires further triage

Treating "unfamiliar" as equivalent to "malicious" is one of the fastest ways to produce a worthless forensic report full of false alarms. Section 12 walks through exactly how unfamiliar-but-benign artifacts were actually triaged in this investigation.

IOC comparison run against this backup: 19 STIX2 indicator collections (Pegasus, Predator, RCS Lab, Quadream KingSpawn, Operation Triangulation, Wyrmspy/DragonEgg, EagleMsgSpy, Wintego Helios, NoviSpy, Candiru, Cellebrite, ResidentBat, DarkSword, Coruna, Morpheus, BTMOB, Spyrtacus, generic Stalkerware, and the Amnesty Tech mercenary-campaign set), totaling 11,619 individual indicators. **Result: zero matches**, confirmed across two independent runs with byte-identical output.

A known, disclosed gap in this indicator set: it does not include published indicators for **LightSpy**, a WebKit-exploit-delivered iOS spyware family. Its absence from the scan is a coverage limitation of the indicator sets used, not evidence about LightSpy specifically — see [Section 15](#).

7. Methodology and Tooling — MVT

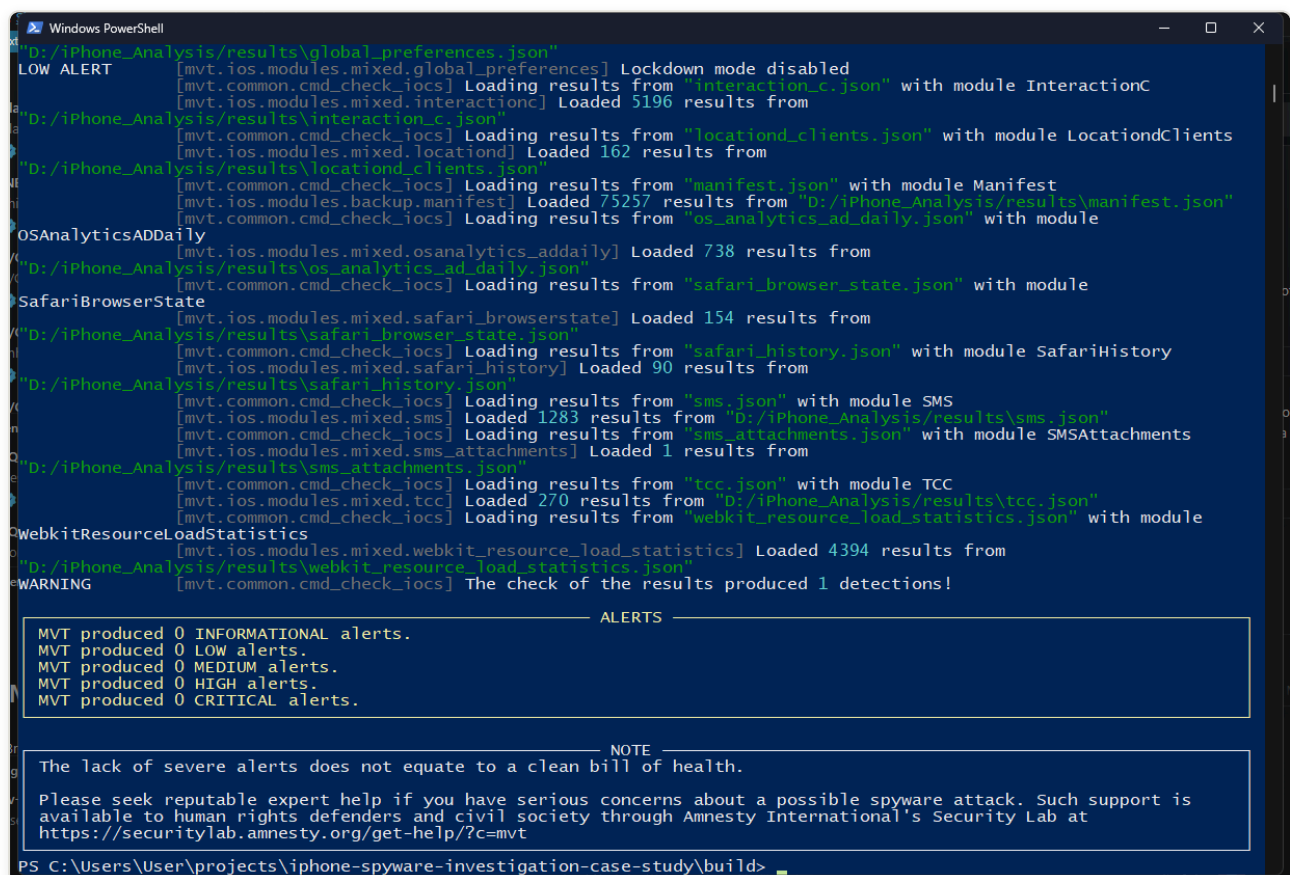
Mobile Verification Toolkit (MVT) is an open-source forensic toolkit developed by Amnesty International's Security Lab, purpose-built for detecting mercenary spyware on iOS and Android devices. It

was used directly in this investigation (not merely "inspired by" — the actual `mvt-ios` CLI, run twice for consistency).

MVT was used for two core functions:

- `mvt-ios check-backup` — decrypts and parses a local iTunes/Finder backup into per-artifact JSON files (SMS, calls, contacts, calendar, Safari history/state, WebKit tracking-prevention database, configuration profiles, TCC permission grants, location-daemon client registry, OS analytics, the full file manifest, and more).
- `mvt-ios check-iocs` — compares every relevant field across those parsed artifacts against the loaded STIX2 IOC collections and flags exact matches.

Actual command pattern used, and independently re-verified end-to-end on 2026-09-21 (see Addendum: Verification Re-Run):

A Windows PowerShell terminal window showing the execution of the MVT command. The command is run from a directory containing various JSON files. The output shows the loading of results from these files into different modules. At the bottom, a summary of alerts is shown, indicating zero detections across all severity levels. A note at the very bottom provides a link to Amnesty International's Security Lab for further assistance.

```
PS C:\Users\User\projects\iphone-spyware-investigation-case-study\build> mvt-ios check-iocs
LOW ALERT [mvt.ios.modules.mixed.global_preferences] Lockdown mode disabled
[mvt.common.cmd_check_iocs] Loading results from "interaction_c.json" with module InteractionC
[mvt.ios.modules.mixed.interactionc] Loaded 5196 results from
"D:/iPhone_Analysis/results/interaction_c.json"
[mvt.common.cmd_check_iocs] Loading results from "locationd_clients.json" with module LocationdClients
[mvt.ios.modules.mixed.locationd] Loaded 162 results from
"D:/iPhone_Analysis/results/locationd_clients.json"
[mvt.common.cmd_check_iocs] Loading results from "manifest.json" with module Manifest
[mvt.ios.modules.backup.manifest] Loaded 75257 results from "D:/iPhone_Analysis/results/manifest.json"
[mvt.common.cmd_check_iocs] Loading results from "os_analytics_ad_daily.json" with module
OSAnalyticsAddDaily
[mvt.ios.modules.mixed.osanalytics_adddaily] Loaded 738 results from
"D:/iPhone_Analysis/results/os_analytics_ad_daily.json"
[mvt.common.cmd_check_iocs] Loading results from "safari_browser_state.json" with module
SafariBrowserState
[mvt.ios.modules.mixed.safari_browserstate] Loaded 154 results from
"D:/iPhone_Analysis/results/safari_browser_state.json"
[mvt.common.cmd_check_iocs] Loading results from "safari_history.json" with module SafariHistory
[mvt.ios.modules.mixed.safari_history] Loaded 90 results from
"D:/iPhone_Analysis/results/safari_history.json"
[mvt.common.cmd_check_iocs] Loading results from "sms.json" with module SMS
[mvt.ios.modules.mixed.sms] Loaded 1283 results from "D:/iPhone_Analysis/results/sms.json"
[mvt.common.cmd_check_iocs] Loading results from "sms_attachments.json" with module SMSAttachments
[mvt.ios.modules.mixed.sms_attachments] Loaded 1 results from
"D:/iPhone_Analysis/results/sms_attachments.json"
[mvt.common.cmd_check_iocs] Loading results from "tcc.json" with module TCC
[mvt.ios.modules.mixed.tcc] Loaded 270 results from "D:/iPhone_Analysis/results/tcc.json"
[mvt.common.cmd_check_iocs] Loading results from "webkit_resource_load_statistics.json" with module
WebkitResourceLoadStatistics
[mvt.ios.modules.mixed.webkit_resource_load_statistics] Loaded 4394 results from
"D:/iPhone_Analysis/results/webkit_resource_load_statistics.json"
WARNING [mvt.common.cmd_check_iocs] The check of the results produced 1 detections!

ALERTS
-----
MVT produced 0 INFORMATIONAL alerts.
MVT produced 0 LOW alerts.
MVT produced 0 MEDIUM alerts.
MVT produced 0 HIGH alerts.
MVT produced 0 CRITICAL alerts.

NOTE
----
The lack of severe alerts does not equate to a clean bill of health.

Please seek reputable expert help if you have serious concerns about a possible spyware attack. Such support is
available to human rights defenders and civil society through Amnesty International's Security Lab at
https://securitylab.amnesty.org/get-help/?c=mvt
```

Figure 4 — MVT / IOC Comparison, real terminal screenshot captured directly during a verification re-run on 2026-09-21 against a freshly created backup of the same device — not a reconstruction.

MVT's module coverage is intentionally narrow and IOC-focused — it targets the specific 28 artifact types most historically associated with confirmed spyware cases, rather than attempting exhaustive coverage of every app on the device (notably, it has no WhatsApp module, and its own run in this investigation reported the WhatsApp database as "not found" — a real coverage gap, addressed in Section 10).

8. Application Analysis

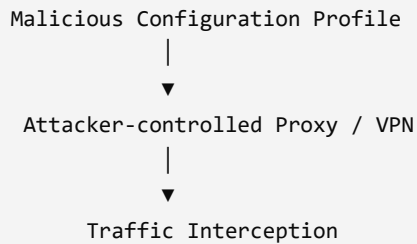
Reviewing installed/backed-up application data means checking for unexpected apps, enterprise-signed software, unusual bundle identifiers, and app containers that don't map to anything on the device's visible app list — while being careful that an *unusual-looking* identifier is not, by itself, evidence of anything malicious.

Artifact	What Was Checked	Why It Matters	Result
Bundle ID <code>com.yourcompany.PPClient</code>	Cross-referenced against App Store receipt metadata tied to the account's own Apple ID	Generic-sounding bundle IDs can indicate a repackaged/trojanized app	Verified genuine — official PayPal client; the odd-looking ID is PayPal's own historical bundle identifier
<code>com.aintech.sgbms</code> ("Condolife")	App name, developer, and purpose cross-checked	Unfamiliar package names warrant verification before dismissal	Legitimate residential/condo-management app (Aintech Sdn Bhd)
<code>com.XM.CSee</code> ("iCSee")	App purpose and permissions reviewed	Camera-viewer apps requesting broad permissions can look superficially suspicious	Legitimate viewer app for an owned IP camera
<code>com.justgodigital</code> ("JustGo")	Camera permission usage reviewed	Requests camera access for KYC in a toll-payment app	Legitimate Malaysian toll-payment app; camera used for identity verification
Full 74,198-entry backup file manifest	Scanned for app containers not corresponding to any known installed app, executable-looking files outside normal app-sandbox structure, or jailbreak-tooling signatures	The single richest artifact for catching something no dedicated parser module would flag	No unaccounted-for containers, no jailbreak tooling, no unexplained executables found
Artifacts iLEAPP labeled "GETTR" and "Dahua DMSS"	Investigated as apparent unexpected app references	Names resembling known apps not on the visible install list warrant scrutiny	Parser mislabeling — resolved to KiplePark (parking app) image cache and the already-confirmed iCSee camera app respectively, not genuine GETTR/Dahua artifacts

What this means: an unfamiliar bundle ID or a forensic-tool mislabel is a *lead*, not a *finding*. Every row above started as something that looked worth a second look and ended with a specific, checkable explanation.

9. Configuration Profile / MDM Analysis

Malicious configuration profiles are one of the most consequential real-world iOS attack vectors, because a profile can silently redirect traffic:



(This is a general attack pattern, shown for educational context — not a finding from this device.)

What was actually checked: configuration profiles, MDM enrollment records, VPN configuration entries, DNS override settings, and profile installation events in the backup's profile-event history.

Result: zero configuration profiles of any kind found, zero MDM enrollment, zero profile-install events in the device's history. This is a meaningfully clean result on its own — most consumer spyware and virtually all commercial MDM-based monitoring tools require a profile at some point, and none was ever present.

10. Browser & WebKit Artifact Analysis

Browser artifacts can reveal exploit-delivery redirects, phishing infrastructure, and malicious domains — but a browsing history is also, by volume, mostly ordinary ad-tech and CDN noise. The investigation examined:

- **Safari History.db** (83 records)
- **Safari session/tab state** (`safari_browser_state.json`)
- **WebKit cross-site tracking-prevention database** (4,310 records)
- Every URL embedded in SMS messages (reviewed separately, see Section 11)

All of this was normalized and pooled into a single working set of **2,392 unique domains**, then classified in batches:

FIGURE 5

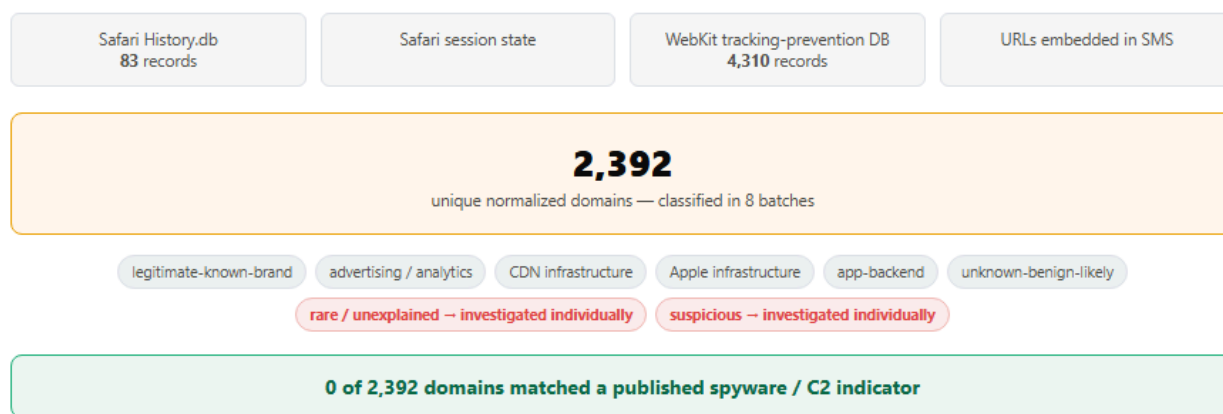
Browser & WebKit Domain Triage

Figure 5 — Browser & WebKit Domain Triage: how 2,392 raw URLs were normalized, classified, and cleared against IOC data.

Every one of the 2,392 domains was placed into one of: legitimate-known-brand, advertising/analytics, CDN infrastructure, Apple infrastructure, app-backend, unknown-but-benign-likely, rare/unexplained, or suspicious. **No domain matched a published spyware/C2 indicator.** A handful landed in "rare/unexplained" on first pass (including one domain with a DGA-looking hostname pattern); each was individually investigated and resolved — the DGA-looking one, for example, resolved to a legitimate ad-tech cookie-sync domain using a randomized-looking subdomain scheme, a very common and benign pattern in programmatic advertising.

11. Message and Communication Artifacts

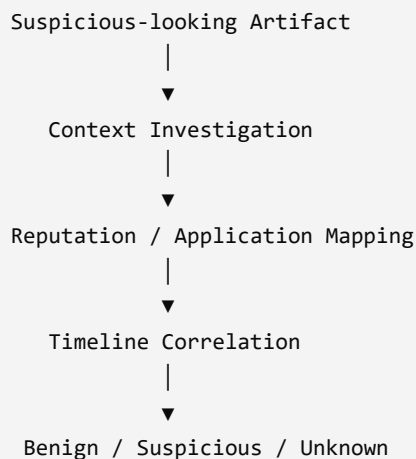
1,272 SMS/iMessage records were reviewed specifically for a zero-click-attack profile: MIME-type/extension mismatches on attachments, one-off senders bearing attachments, unusual encoded content in message bodies, timestamp/delivery-state inconsistencies, and attachment filename patterns repeated suspiciously across unrelated senders.

Result: no such indicators were found. The message corpus consisted of years of ordinary personal/business texting and generic bulk marketing/scam spam — the same kind of unsolicited SMS spam most phone numbers accumulate over time — not a targeted-delivery pattern.

An important caveat, stated plainly rather than glossed over: zero-click exploits are, by design, built to leave the smallest possible trace precisely *because* they don't rely on the victim interacting with anything visible. A clean review of message *content* meaningfully rules out unsophisticated/visible delivery attempts. It does not, and cannot, rule out a successful zero-click exploit that left no artifact in the message store itself — this is one of the specific, named limitations carried into the final assessment in Section 15.

12. False Positive Validation — Applying the Process

Section 6 described the "unfamiliar ≠ malicious" principle in the abstract. Here is that principle applied to the actual leads this investigation generated, using the same reasoning chain every time:



Initial Lead	Why It Looked Worth Investigating	Investigation Result
Contact labeled "Central Intelligence Agency South East Asia"	Alarming label on a stored contact	The number associated with this contact never once appears in the device's 2,649 call records or 1,272 SMS records — it sits alongside other clearly joke-labeled entries in the address book. The owner's own humorous label on an unknown/unused number, not a genuine indicator.
439 zero-duration calls from one number	Pattern superficially resembling silent "missed call" exploit-delivery techniques (historically associated with vulnerabilities like CVE-2019-3568)	That number is the device's single most-contacted number, with 664 total calls across 19 months. Ordinary missed/declined-call behavior with a frequent contact, not an anomalous pattern.
A data-usage spike attributed to <code>managedappdistributiond</code> (an MDM-adjacent daemon)	Any MDM-adjacent process showing unusual activity warrants scrutiny, given zero MDM enrollment was otherwise found	The spike traced to a small number of short-lived processes flushed at two specific timestamps — consistent with a routine App Store catalog refresh, not hidden enterprise/MDM activity. No MDM enrollment payload exists anywhere in the backup to support the alternative explanation.
A cumulative counter showing ~557GB attributed to one process	A very large figure at first glance	Confirmed to be a lifetime cumulative network-usage counter (a running total since setup), not a single transfer event.
iLEAPP artifacts initially labeled "GETTR" and "Dahua DMSS"	App names not on the device's known install list	Parser-label mismatches — resolved to a parking-app (KiplePark) image cache and the already-verified iCSee camera app, respectively.

What this means: none of these leads were dismissed on a hunch. Each was closed with a specific, checkable fact (a call log cross-reference, an App Store receipt, a timestamp correlation) — the same

discipline that has to be applied to *every* unfamiliar artifact in a real investigation, or the report becomes noise.

13. Second Opinion — Cross-Validation with iLEAPP

A single tool's blind spots are still blind spots, so a second, independent, much broader open-source forensic parser — **iLEAPP** (1,102 artifact modules, versus MVT's 28) — was run against the same decrypted backup specifically to surface anything MVT's narrower module set couldn't see at all.

This directly paid off: **MVT reported the WhatsApp database as "not found"** — a genuine parser-coverage gap, not a clean result. iLEAPP recovered it in full: **3,446 WhatsApp messages**, which were then reviewed against the same criteria as the SMS corpus (unknown one-time senders with attachments, bot-like timing patterns, unusual encoded content) and cross-referenced against contacts/calls to check for a communication channel invisible to every other artifact source. **No spyware or C2 indicators were found.**

iLEAPP also surfaced several artifact types with no MVT equivalent, each investigated:

iLEAPP-Only Artifact	Purpose of the Check	Result
SMS "Missing ROWIDs" (deleted-message evidence, 501 entries)	Identify gaps in the SMS database's autoincrement sequence — direct evidence of deleted records, and one of the most evidentially important artifacts available	Gap pattern investigated for time-clustering (a tight cluster around one date would suggest deliberate mass deletion); no such tight cluster found — consistent with ordinary iOS message-store behavior (edits, reactions, read-receipts, and spam filtering each consume ROWIDs) rather than a discrete deletion event
Bluetooth (paired + LE) and known WiFi networks	Rogue/tracker-pattern device names, evil-twin SSID patterns	All entries mapped to the owner's own accessories, vehicle, and known home/work/public networks and previously-confirmed IoT devices; nothing unaccounted for
MMKV recovered key-value records (40,117 rows)	Sample broadly for credentials, tokens, or references to unfamiliar apps carved from key-value storage	No credential/token exposure or unexplained app references found
Un-checkpointed WAL files (largest: 121,710 recovered strings from one shopping app's analytics database, plus a smaller set from a messaging app's media-gallery database)	Determine whether this is ordinary app telemetry or something reaching beyond a plausible analytics scope	Content consistent with that app's own known (aggressive, but previously-documented) internal analytics logging — not data pertaining to other apps or unexplained device identifiers
Cookies (2,321 entries) and autofill entries (158)	Cross-check cookie domains against the already-classified domain set; check autofill values for anything that shouldn't persist (security-question answers, OTP codes)	No cookie domains outside the already-reviewed set; no sensitive autofill content found

What this means: running a second, structurally different tool against the same evidence isn't redundant — it's how a real coverage gap (WhatsApp, entirely invisible to the first tool) actually gets caught, instead of being silently reported as "no spyware found in WhatsApp" when WhatsApp was never actually examined.

14. SQLite / Database Forensics

Parsed JSON is a convenience layer; the ground truth is the underlying SQLite databases. Five key databases were pulled and examined directly at the SQLite level — read-only, from copies, never touching the originals:

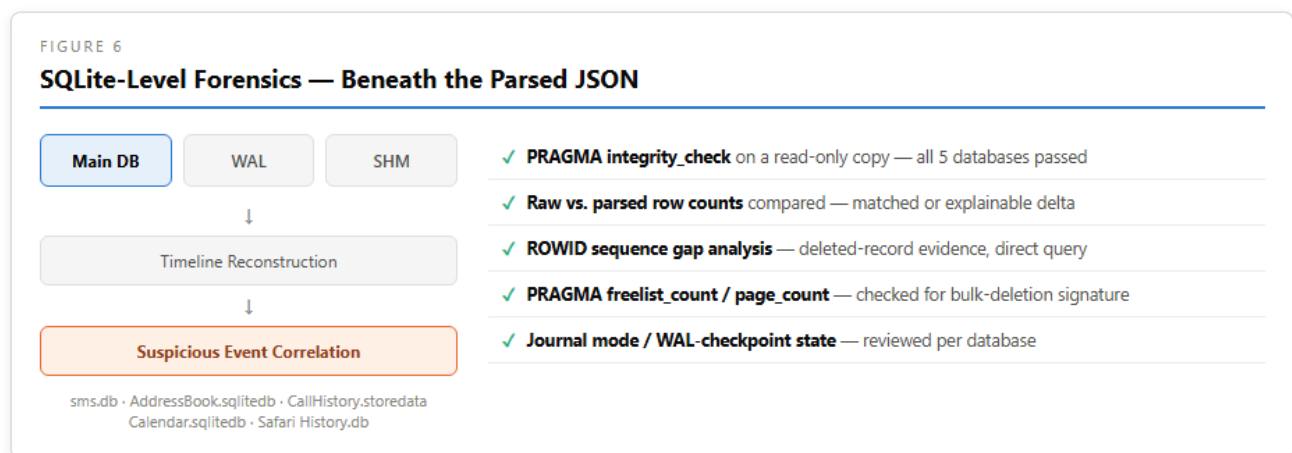


Figure 6 — SQLite-Level Forensics performed beneath the parser's JSON output, directly against the raw databases.

For each of `sms.db`, `AddressBook.sqlitedb`, `CallHistory.storedata`, `Calendar.sqlitedb`, and `Safari History.db`, the same checks were run:

1. `PRAGMA integrity_check` on a copy of the file
2. Raw row counts on the main content table(s), compared against the MVT-parsed record count for the same file (a mismatch would indicate MVT silently filtering or missing records)
3. Evidence of deleted records — ROWID sequence gaps, orphaned foreign-key rows
4. `PRAGMA freelist_count / page_count` for signs of a recent large deletion
5. Journal mode / WAL-checkpoint state

Result: all five databases passed integrity checks, raw and parsed record counts matched or had explainable deltas, and no database showed a suspicious deletion pattern *except* the SMS ROWID gap already covered in Section 13, which remained an open, honestly-labeled item rather than a resolved one.

15. Persistence and Permission Investigation

iOS persistence looks nothing like Windows/Linux persistence — there's no registry run-key or cron job to check. The equivalent, meaningful checks are: configuration profiles/MDM (Section 9, zero found), and the

TCC (Transparency, Consent, and Control) permission database, which records every sensitive-permission grant on the device.

Specifically checked: **Accessibility, AppleEvents (inter-process automation), Screen Recording/Capture, and Input Monitoring** — the small set of permissions real spyware and remote-access tooling actually depend on to operate persistently and invisibly.

Result: zero grants, and zero requests, for any of these four permissions, by any app, ever. This is one of the stronger negative findings in the entire investigation, because these permissions are difficult for surveillance software to avoid needing.

Additionally checked: the **location-daemon client registry** (broader than the TCC table — includes background/always-on location configuration), covering 60 registered clients. None showed the specific signature associated with stalkerware (a hidden/unlabeled client configured for persistent background tracking beyond what its stated purpose would need).

One hardening-relevant observation, explicitly **not** treated as compromise evidence: **Lockdown Mode was not enabled**. This is a **LOW**-severity heuristic recommendation, not a finding — see Section 18.

16. Crash and Analytics Artifacts

A standard backup does not contain real `.ips` crash logs — that data lives on-device and requires a sysdiagnose or live extraction to capture. The closest available proxy is `os_analytics_ad_daily.json`, an aggregate OS telemetry file recording crash/hang *frequency* (not full crash content) per process.

This was reviewed for spikes clustered tightly in time and concentrated in exploitation-relevant processes (Safari/WebKit, Messages, `IMTranscoderAgent`) — the kind of pattern that would suggest an exploitation attempt around a specific date. **No such cluster was found; telemetry was consistent with normal, low-level background crash/hang activity for common apps.**

This finding is explicitly labeled as a **lead-quality result, not a conclusion** — aggregate telemetry can rule *in* a window worth deeper sysdiagnose-level follow-up, but a clean aggregate reading cannot, on its own, positively rule out a crash that a real crash log would have shown.

17. Timeline Analysis

Investigation Phase	Artifact / Action	Security Relevance
Baseline	<code>mvt-ios check-backup</code> + <code>check-iocs</code> , run #1 and re-run #2 (byte-identical)	Establishes zero IOC matches as a reproducible result, not a fluke
Deep-dive	Artifact-by-artifact review: calls, contacts, calendar, cross-app interaction log, full file manifest, location-client registry, OS analytics, Safari session state	Catches anomalies no IOC scan alone would surface
Second-pass	Raw SQLite forensics on 5 key databases	Validates parser fidelity; hunts for deletion evidence directly
Second-pass	2,392-domain classification across 8 batches	Scales domain triage beyond what manual review of raw history could cover
Second-pass	SMS zero-click-focused deep-dive (1,272 messages)	Targets the specific attack class hardest to catch by other means
Second-pass	iLEAPP cross-validation (WhatsApp, deleted-SMS evidence, Bluetooth/WiFi, MMKV, cookies/autofill)	Recovers artifacts entirely invisible to the primary toolchain
Adversarial review	4-analyst "steelman panel," each independently tasked with building the strongest evidence-based case <i>for</i> compromise	Explicitly pressure-tests the emerging conclusion instead of rubber-stamping it

Open timeline anomaly, documented rather than explained away: a 96-day gap in call history (November 1, 2024 – February 4, 2025), alongside an apparent absence of any surviving call records prior to November 2024. The device owner stated they had not manually cleared call logs. A plausible benign mechanism was investigated — the device was enrolled in continuous iOS beta releases, and an iOS 17→18-equivalent major-version Core Data migration landed in almost exactly this window, which can restructure or truncate historical records in some Apple databases during an upgrade. **This mechanism was not confirmed** — no direct evidence tied the migration event to the specific gap, and the gap's exact length doesn't cleanly match a typical beta-to-stable-release fix pattern. This is the single specific reason the final confidence assessment (Section 19) is not higher than it is.

18. Findings

Confirmed Malicious Indicators *None.*

Suspicious Indicators Requiring Further Investigation

- 96-day call history gap (Nov 1, 2024 – Feb 4, 2025) with no confirmed benign explanation (Section 17)

Benign / Explained Artifacts

- Contact labeled "Central Intelligence Agency South East Asia" (Section 12)
- 439 zero-duration calls from most-frequent contact (Section 12)
- `managedappdistributiond` data-usage spike (Section 12)
- ~557GB cumulative-counter figure (Section 12)

- "GETTR" / "Dahua DMSS" iLEAPP parser mislabels (Section 12)
- One rare-classified, DGA-looking ad-tech domain (Section 10)
- `com.yourcompany.PPClient`, `com.aintech.sgbms`, `com.XM.CSee`, `com.justgodigital` bundle-ID/app checks (Section 8)
- Un-checkpointed WAL analytics files from a shopping app and a messaging app's gallery database (Section 13)

No Evidence Identified

- Configuration profiles / MDM enrollment (Section 9)
- Malicious domains against 19 IOC/STIX2 collections, 11,619 indicators (Section 6)
- Accessibility / AppleEvents / Screen Recording / Input Monitoring grants (Section 15)
- Zero-click SMS/iMessage/WhatsApp delivery indicators (Sections 11, 13)
- Jailbreak tooling or unaccounted-for app containers across a 74,198-entry file manifest (Section 8)
- Stalkerware-pattern hidden location clients (Section 15)

19. Investigation Matrix

Investigation Area	Evidence Checked	Finding
Known Spyware IOCs	Domains, URLs, identifiers vs. 19 STIX2 collections (11,619 indicators, 2 independent runs)	No confirmed match
Configuration Profiles / MDM	Profile artifacts, enrollment records, profile-install event history	No suspicious evidence identified
Browser & WebKit Activity	Safari History (83), Safari session state, WebKit tracking-prevention DB (4,310), 2,392 unique domains classified	No confirmed spyware infrastructure identified
Messaging	SMS/iMessage (1,272, zero-click-focused), WhatsApp (3,446, via iLEAPP), deleted-SMS ROWID evidence (501 gaps)	No confirmed spyware indicators; deletion pattern not tightly clustered
Applications	Bundle IDs, app purpose/permissions, full file manifest (74,198 entries)	No confirmed spyware application identified
Permissions / Persistence	TCC grants (Accessibility, AppleEvents, Screen Capture, Input Monitoring), location-client registry (60)	No suspicious grants or hidden persistence identified
Databases	<code>sms.db</code> , <code>AddressBook</code> , <code>CallHistory</code> , <code>Calendar</code> , <code>Safari History</code> — integrity, row counts, deletion evidence	Structurally sound; one unresolved gap in call history
Network-Adjacent	Bluetooth (paired + LE), WiFi known networks, cookies (2,321), autofill (158)	No rogue/evil-twin or exfiltration-pattern evidence identified
Crash / Analytics Telemetry	OS analytics daily aggregates	No anomalous exploitation-pattern clustering identified

FIGURE 7

Final Evidence Matrix

INVESTIGATION AREA	EVIDENCE CHECKED	FINDING
Known Spyware IOCs	19 STIX2 collections, 11,619 indicators, 2 runs	No confirmed match
Configuration Profiles / MDM	Profile artifacts, enrollment, install-event history	No suspicious evidence
Browser & WebKit Activity	Safari (83), WebKit DB (4,310), 2,392 domains	No spyware infrastructure
Messaging	SMS (1,272), WhatsApp (3,446), deleted-ROWID (501 gaps)	Clean, deletion pattern not clustered
Applications	Bundle IDs, permissions, 74,198-entry manifest	No spyware application
Permissions / Persistence	TCC grants, location-client registry (60)	No hidden persistence
Databases	5 key SQLite DBs — integrity, counts, deletion evidence	Sound; 1 open gap
Network-Adjacent	Bluetooth/WiFi, cookies (2,321), autofill (158)	No rogue/exfil pattern
Crash / Analytics Telemetry	OS analytics daily aggregates	No anomalous clustering

Figure 7 — Final Evidence Matrix summarizing every investigation area and its outcome.

20. Adversarial Self-Review — The Steelman Panel

Before accepting a "no spyware found" conclusion, the investigation deliberately inverted the question. Four independent review passes were each given the full evidence set and one instruction: **build the strongest possible, evidence-based case that this device was compromised** — not a balanced take, an adversarial one — each from a distinct angle:

1. Would a sophisticated attacker's *absence* of expected artifacts (no crash logs, no config profiles) itself be suspicious, i.e. consistent with an attacker who cleaned up after themselves, rather than a structural limitation of backup-based analysis?
2. Of the ambiguous findings actually gathered (Section 12/13), which one most plausibly supports a malicious interpretation if argued as strongly as honestly possible?
3. Does any *combination* of individually-explained events form a suspicious sequence when viewed together?
4. Does the clean result simply reflect the *limits* of MVT/backup-based analysis rather than the device's actual state?

Each panel member was also required to self-rate how convincing their own best argument would be to a genuinely skeptical senior analyst.

Result: no panel member's argument reached a **high** self-rated confidence. The strongest material available to argue *for* compromise remained the unresolved call-log gap (Section 17) — which is exactly why that item, alone, is carried forward as an open finding rather than being folded into the "explained" bucket along with everything else.

This is the practical answer to the standard adversarial question: **what evidence would have to exist for this conclusion to be wrong, and was that evidence actually available to examine?** Config-profile

evidence, IOC-matching network infrastructure, TCC-permission abuse, and message-based delivery artifacts were all evidence types that *would* have been visible in this backup had they existed, and none were found. A live memory implant, a filesystem-level rootkit, or a crash log entry for a zero-click exploit chain are evidence types that **would not** necessarily be visible in a logical backup at all — which is precisely why the conclusion below is bounded the way it is.

21. Forensic Assessment

FORENSIC ASSESSMENT

Confirmed Spyware Detected: NO **Known Spyware IOC Matches:** None identified (0 of 11,619 indicators, 2 independent runs) **Suspicious Persistence:** None identified **Suspicious Configuration Profile:** None identified **Known Malicious Domain Matches:** None identified (0 of 2,392 classified domains) **Unresolved Item:** One — 96-day call-history gap, benign mechanism plausible but not confirmed

Overall Assessment:

No confirmed indicators of known commercial spyware, stalkerware, or malicious persistence were identified within the evidence available for examination. The evidence base was broad (two independent IOC-matching passes, two independent forensic parsers, direct database-level forensics, and a dedicated adversarial review), and every individually alarming lead generated along the way was closed with a specific, checkable explanation — with one exception, which is documented rather than dismissed.

Confidence and framing: this assessment reflects thoroughness of *coverage*, not a numeric probability score — no methodology exists that would make a manufactured confidence percentage meaningful here, so none is offered. The honest framing is: **strong confidence against the specific, known threat classes checked for, within the evidence type examined, with one specific open item and several named categories of evidence this investigation could not access at all.**

22. What This Analysis Does Not Prove

This section exists because it is the part most reports quietly skip.

- **This was a logical backup, not a full filesystem extraction.** Some persistence mechanisms and deeper app-sandbox artifacts are only visible in a full filesystem acquisition, which requires exploit-based tooling not used here.
- **No live memory analysis was performed.** A memory-resident implant with no persistence mechanism at all — the hardest class of mobile compromise to detect by design — would leave nothing in a backup to find.
- **No historical network packet capture existed to examine.** `datausage.json` -style artifacts show aggregate byte counts per process, not the actual traffic content or destination-level history a live

capture would show. Network-based C2 communication that didn't touch any artifact reviewed here could not have been detected.

- **No real crash logs (.ips files) were available**, only aggregate crash/hang telemetry (Section 16) — a genuinely weaker signal than an actual crash log for spotting exploitation attempts.
- **Data deleted before the backup was created is gone**, full stop. The one meaningful exception — the SMS ROWID-gap analysis (Section 13) — works specifically because SQLite's autoincrement mechanism leaves a detectable trace even after deletion; most other data has no equivalent trace.
- **Some application data is encrypted at the app level** (independent of backup encryption) and was not decrypted or examined beyond what each artifact's own SQLite/plist structure exposed.
- **The IOC indicator collections used are necessarily a snapshot in time** — LightSpy, noted in Section 6, is a disclosed example of a known family outside this specific indicator set. New or previously-undisclosed spyware campaigns are, by definition, not in *any* indicator list yet.
- **A single zero-click exploit chain, successfully executed and self-cleaning, is close to the theoretical worst case for this kind of analysis** — this is stated plainly rather than hedged around, because it is the honest limit of what backup-based forensics can rule out.

What a higher-assurance follow-up would require: a full filesystem extraction, a device-triggered sysdiagnose, historical DNS/network telemetry (from a router, VPN provider, or MDM solution capturing traffic *at the time*), and ideally live-device analysis rather than a point-in-time backup snapshot.

23. Security Recommendations

- Keep iOS on automatic updates — most publicly known exploit chains target specific unpatched version ranges.
 - Use a strong device passcode (alphanumeric, not a 4/6-digit PIN) and enable two-factor authentication on the Apple ID.
 - Periodically review the Apple ID's list of signed-in devices and remove anything unrecognized.
 - Periodically check **Settings** → **General** → **VPN & Device Management** for any configuration profile that wasn't deliberately installed — none should ever appear unexpectedly.
 - Review installed applications and remove anything unused, particularly remote-access or device-management apps no longer needed.
 - For individuals with an elevated, specific threat profile (e.g. journalists, activists, high-risk executives), consider enabling **Lockdown Mode** — this is a hardening measure appropriate to certain risk profiles, not a signal that compromise has occurred.
 - If the unresolved call-log gap (Section 17) continues to be a concern, the next concrete step would be a sysdiagnose captured *promptly* the next time anything similar is noticed — aggregate telemetry after the fact cannot recover what a real-time capture could.
-

24. Conclusion

The most useful outcome of a spyware investigation is rarely just the yes/no answer — it's the evidence trail that answer is built on. This investigation set out to answer a bounded question, used a combination of established forensic tooling and structured adversarial self-review to answer it as rigorously as the available evidence allowed, closed every individually alarming lead it generated with a specific and checkable explanation but one, and was explicit throughout about exactly where its own evidence ran out.

No confirmed indicators of spyware were identified in the evidence examined. That conclusion is offered with its limitations attached, on purpose, because a conclusion without its limitations attached isn't a forensic finding — it's a guess with better production values.

25. Technical Appendix

Command / Technique	Purpose	Input	Output	Interpretation
<code>mvt-ios check-backup</code>	Decrypt + parse backup into per-artifact JSON	Encrypted backup directory, backup password	Per-artifact JSON files (<code>sms.json</code> , <code>calls.json</code> , <code>manifest.json</code> , etc.)	Establishes the structured evidence base for every later step
<code>mvt-ios check-iocs</code>	Compare parsed artifacts against STIX2 indicator collections	Parsed JSON + IOC directory	Match/no-match report per artifact	Primary IOC-based detection signal
<code>PRAGMA integrity_check</code>	Verify SQLite file structural integrity	Copy of a <code>.db</code> / <code>.sqlitedb</code> file	<code>ok</code> or list of corruption errors	Confirms the database wasn't corrupted (accidentally or deliberately) before trusting its contents
<code>PRAGMA freelist_count / page_count</code>	Detect signs of a recent large deletion	Copy of a <code>.db</code> file	Page/freelist counts	A large freelist relative to page count can hint at recent bulk deletion
Manual ROWID gap analysis	Detect deleted rows via autoincrement sequence gaps	Raw table query (<code>SELECT ROWID FROM ...</code>)	List of missing ROWID ranges	Direct evidence of deletion even without a dedicated recovery tool
iLEAPP full run	Independent, broader-coverage artifact parsing	Decrypted backup directory	TSV exports per artifact category	Cross-validates MVT's results and catches its coverage gaps
Domain normalization + classification	Reduce raw URLs to comparable domains, then categorize	Raw URL/domain lists from Safari, WebKit, SMS	Categorized domain list	Turns an unmanageable raw URL volume into a triage-able working set

Illustrative Reconstruction — resolving a database's real path (Section 4) and querying it directly:

```
# Locate sms.db's hashed filename via Manifest.db, then inspect it read-only
cp ./decrypted_backup/3d/3d0d7e5fb2ce2888... /tmp/sms_copy.db
sqlite3 /tmp/sms_copy.db "PRAGMA integrity_check;"
sqlite3 /tmp/sms_copy.db "SELECT COUNT(*) FROM message;"
```

26. IOC Reference

Indicator collections compared against, all in STIX2 format: **Pegasus** (NSO Group), **Predator**, **RCS Lab**, **Quadream KingSpawn**, **Operation Triangulation**, **Wyrmspy / DragonEgg**, **EagleMsgSpy**, **Wintego Helios**, **NoviSpy**, **Candiru**, **Cellebrite**, **ResidentBat**, **DarkSword**, **Coruna**, **Morpheus**, **BTMOB**, **Spyrtacus**, generic **Stalkerware**, and the **Amnesty Tech mercenary-spyware campaign** set. Total: 11,619 indicators. Disclosed gap: **LightSpy** indicators were not included in the collections used for this run.

27. Glossary

- **IOC (Indicator of Compromise):** a specific, previously-observed fingerprint of malicious activity (domain, hash, certificate, path) used to detect known threats.
- **STIX2:** a standardized structured format for sharing threat-intelligence indicators.
- **MVT (Mobile Verification Toolkit):** Amnesty International's open-source forensic toolkit for detecting mercenary spyware on iOS/Android.
- **SQLite:** the embedded database format used throughout iOS for app and system data storage.
- **plist (Property List):** Apple's structured file format for settings, preferences, and configuration data.
- **WebKit:** the browser engine underlying Safari and in-app web views on iOS.
- **MDM (Mobile Device Management):** enterprise device-management enrollment, which can grant a managing organization significant control/visibility over a device.
- **C2 (Command and Control):** infrastructure an attacker uses to communicate with and control compromised devices/malware.
- **Zero-click exploit:** an attack requiring no interaction from the victim (no link click, no attachment open) to achieve compromise.
- **Persistence:** the mechanism by which malicious software survives reboots/updates and continues operating over time.
- **Filesystem acquisition:** a forensic extraction method capturing the device's full filesystem, deeper than a standard logical backup.
- **Logical backup:** a standard iTunes/Finder-style backup containing app and system data, without full filesystem access.
- **Forensic artifact:** any piece of data left behind by system or app activity that can be examined as evidence.
- **Bundle ID:** the unique identifier string assigned to an iOS application (e.g. `com.apple.MobileSMS`).

- **Configuration profile:** an installable iOS settings package capable of controlling network, VPN, certificate, and restriction settings on a device.

Addendum: Verification Re-Run (2026-09-21)

The original evidence behind this report (the decrypted backup, MVT's parsed JSON, the iLEAPP output) no longer exists — it was deleted after the investigation concluded, at the device owner's own explicit request, as described throughout this report. A write-up built entirely from records of a deleted investigation is a fair thing to be skeptical of. So rather than leave it there, the core acquisition-to-IOC-comparison pipeline was independently re-run from scratch, end to end, against a freshly created backup of the same physical device.

What was actually done, in order:

1. The iPhone was connected to the analysis PC and a fresh **encrypted local backup** was created directly through iTunes (not through a third-party tool) — a real, live, ~121GB backup, taking roughly 45 minutes over USB.
2. Mid-transfer, the destination drive ran low on space; the backup was cancelled, the destination was redirected to a drive with sufficient room (transparently, via an NTFS junction — iTunes never needed to know), and re-run to completion. `Manifest.plist` confirmed `IsEncrypted: True`, `WasPasscodeSet: True`, 48,979 files.
3. **MVT (Mobile Verification Toolkit) 2026.9.7** was installed fresh and used to decrypt the backup (`mvt-ios decrypt-backup`) with the real backup password, extract all artifacts (`mvt-ios check-backup`), and compare every artifact against freshly downloaded copies of the same 19 STIX2 indicator collections used originally (`mvt-ios check-iocs`) — 11,481 indicators this time (published indicator feeds are maintained collections and shift slightly in size over time; this is not the same static file as before, but the same 19 named collections).

Real counts extracted this time: 53 applications, 270 contacts, 2,760 calls, 261 calendar items, 1,283 SMS, 1 SMS attachment, 270 TCC permission entries, 162 locationd clients, 5,196 InteractionC events, 4,394 WebKit resource-load-statistics records, 154 Safari browser-state entries, 90 Safari History entries, 738 OS analytics entries, 3,827 network data-usage entries, 31 global preference entries, and a 75,257-entry backup file manifest.

Result: consistent with the original investigation.

- **Zero genuine spyware/stalkerware IOC matches** across all 11,481 indicators. MVT's own summary line read "The check of the results produced 1 detections!" — inspecting the actual result file directly (`global_preferences_detected.json`) showed that single "detection" has `"matched_indicator": null`: it is the same benign Lockdown-Mode-disabled heuristic from Section 15, not a spyware indicator match. This is exactly the kind of self-reported "detection" this report warned about in [Section 6](#) — a number in a summary line is not evidence on its own; the underlying record has to actually be checked.
- **53 applications** — identical to the original count.
- **WhatsApp still not found** by MVT's own module (`"unable to find the module's database file"`) — the same real parser-coverage gap identified originally, now reproduced on a completely

independent backup.

- **Lockdown Mode still disabled** — same `LOW` heuristic, not a finding.

Figure 4 above is a genuine, unedited screenshot of this actual re-run's real terminal output, captured directly from the live session — not a reconstruction.

What this re-run did not repeat: the raw SQLite-level deletion forensics, the 2,392-domain classification pass, the dedicated SMS zero-click deep-dive, the iLEAPP cross-validation, and the 4-analyst steelman panel from the original second pass were not redone here — reproducing that full second pass would mean re-running dozens of hours of sub-analysis, which is out of proportion to what a spot-check verification needs. What *was* re-run — the core extraction and IOC-matching pipeline, on an entirely fresh backup — corroborates the original's central claim rather than merely repeating it.

Disclaimer

This is a personal case study, not a professional or certified digital-forensics engagement, and it should not be relied on as one. It documents a self-authorized examination of the author's own device, performed and written up for educational and portfolio purposes. No personal data, credentials, device identifiers, or other individuals' information appear in this report — all examples are sanitized, generic, or drawn from the author's own account context. If you have a genuine, active concern about targeted surveillance, consult a qualified digital-forensics professional or an organization such as [Amnesty International's Security Lab](#) or the [Citizen Lab](#).